Laboratorio 02: Crittografia di Base con OpenSSL

Decomposizione Pedagogica, Analisi delle Primitive e Guida alla Risoluzione

Corso di Sicurezza dei Sistemi Informativi
Politecnico di Torino — Prof. Cataldo Basile

Anno Accademico 2025/2026

Indice

1 Scopo Globale del Laboratorio **3**

2 Es. 1: Cifratura Simmetrica **4**
2.1 Esercizio 1.1: Cifratura Simmetrica, Padding e Modalità Operative 4
2.1.1 Comando 1: Creazione e Validazione della Dimensione del Plaintext 4
2.1.2 Comando 2: Cifratura con AES-128 in Modalità CBC 4
2.1.3 Comandi 3–6: Cifratura con DES, RC4, 3DES e ChaCha20 5
2.1.4 Comando 7: Ispezione Comparativa delle Dimensioni dei File Cifrati 5
2.1.5 Comando 8: Cifratura AES senza Padding (-nopad) e Fallimento con Dati Disallineati 6
2.1.6 Comando 9: Decifratura di Tutti i File e Validazione di Integrità 6
2.1.7 Comando 10: Derivazione di Chiave e IV tramite Passphrase (KDF) 7
2.2 Esercizio 1.2: Attacco a Forza Bruta su Cifrario a Flusso (RC4) 8
2.2.1 Comando 1: Esecuzione dello Script di Brute-Force su Messaggio di Testo . . . 8
2.2.2 Comando 2: Tentativo di Brute-Force su File Binario Casuale (bruteforce_enc) 8
2.3 Esercizio 1.3: Benchmark e Valutazione delle Prestazioni Simmetriche 10

3 Es. 2: Cifratura Asimmetrica **12**
3.1 Esercizio 2.1: Generazione di Chiavi RSA ed Ispezione Parametri PKCS#1 12
3.2 Esercizio 2.2: Cifratura e Decifratura Asimmetrica con pkeyutl e Limiti Dimensionali . 14
3.2.1 Dimostrazione del Limite Dimensionale di RSA 14
3.3 Esercizio 2.3: Firma Digitale RSA e Meccanismo di Verifica 15
3.4 Esercizi 2.4 & 2.5: Crittografia su Curve Ellittiche (EC) e Firma ECDSA 15
3.5 Esercizio 2.6: Benchmark Comparativo delle Prestazioni Asimmetriche 17

4 Es. 3: Digest Crittografico **19**
4.1 Esercizio 3.1: Calcolo Digest, Effetto Valanga e Costruzione CBC-MAC 19
4.1.1 Dimostrazione dell'Effetto Valanga (Avalanche Effect) 19
4.1.2 Costruzione di una Funzione Hash con Cifrario a Blocchi (CBC-MAC) 19
4.2 Esercizio 3.2: Analisi delle Collisioni con Hash Ridotto a 4-bit (IHA) 19
4.3 Esercizio 3.3: Benchmark degli Algoritmi di Hash 20

5 Es. 4: Esercizi Opzionali **22**
5.1 Esercizio 4.1: Manomissione Bit-Flipping su Cifrario a Flusso vs a Blocchi 22
5.1.1 Cosa accade applicando il medesimo attacco su AES-128-CBC? 22
5.2 Esercizio 4.2: Vulnerabilità dei Pattern in Modalità ECB vs CBC 23
5.3 Esercizio 4.3: Monitoraggio dell'Integrità Ricorsiva di Alberi di File 23

1 Scopo Globale e Architettura del Laboratorio

Il Laboratorio 02 introduce l'utilizzo pratico e approfondito delle **primitive crittografiche fondamentali** tramite la suite open-source `OpenSSL`. L'obiettivo primario non è soltanto apprendere la sintassi della command line interface (CLI), bensì comprendere a fondo il comportamento matematico, le garanzie di sicurezza, i vincoli implementativi e i trade-off computazionali che governano la cifratura simmetrica, la cifratura asimmetrica, le firme digitali e gli algoritmi di digest.

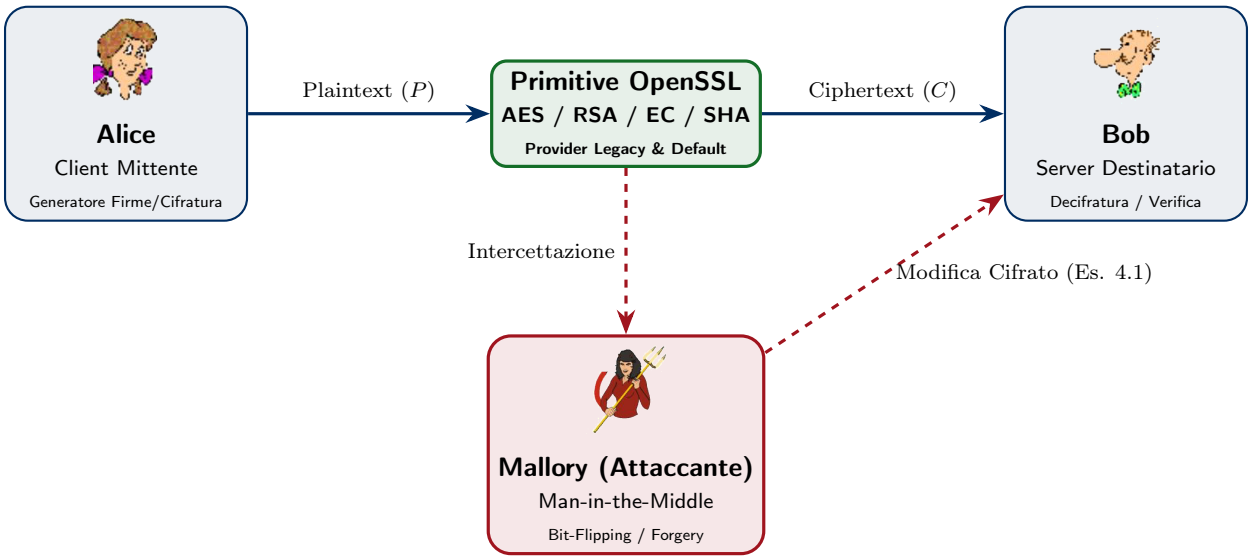


Figura 1: Scenario Architetturale del Laboratorio: Flusso di cifratura tra Alice e Bob e minaccia di manomissione attiva da parte di Mallory.

Obiettivo e Inquadramento Sistemistico

Finalità Didattiche e Competenze Acquisite:

1. **Cifratura Simmetrica:** Gestione esplicita di chiavi (K), vettori di inizializzazione (IV), modalità operative (CBC, ECB) e regole di padding PKCS#7; cifrari a blocchi (AES, DES, 3DES) vs cifrari a flusso (RC4, ChaCha20).
2. **Crittanalisi a Forza Bruta:** Vulnerabilità di spazi di chiavi ridotti (4 bit) e principio dell'Unicity Distance su plaintext con o senza ridondanza nota.
3. **Cifratura Asimmetrica & Firme Digitali:** Anatomia interna di una chiave RSA (PKCS#1 con modulo n, esponente pubblico e, esponente privato d, parametri del Teorema Cinese del Resto $p, q, dP, dQ, qInv$), limite di dimensione $N - 11$ byte, curve ellittiche (ECDSA su curva secp192k1).
4. **Funzioni di Hash:** Proprietà di indistinguibilità, Effetto Valanga (*Avalanche Effect*), costruzione di CBC-MAC, attacchi alla prima pre-immagine e paradosso del compleanno per la ricerca di collisioni.
5. **Integrità e Manomissione Operativa:** Vulnerabilità al Bit-Flipping dei cifrari a flusso privi di autenticazione e controllo di integrità gerarchico di alberi di directory.

2 Esercizio 1: Algoritmi di Cifratura Simmetrica e Forza Bruta

2.1 Esercizio 1.1: Cifratura Simmetrica, Padding e Modalità Operative

Nel primo esercizio si sperimenta la cifratura simmetrica attraverso il modulo `openssl enc`. Viene creato un messaggio in chiaro esattamente pari a 32 byte:

```
Questo e' il mio grande segreto\n
```

Questa precisa dimensione (31 caratteri ASCII più il terminatore di riga) corrisponde a due blocchi esatti di 16 byte per algoritmi moderni come AES ($2 \times 16 = 32$), o quattro blocchi da 8 byte per DES e 3DES ($4 \times 8 = 32$).

2.1.1 Comando 1: Creazione e Validazione della Dimensione del Plaintext

Alice crea il file di testo `ptext` e ne verifica l'esatta occupazione in byte tramite `wc -c`.

Alice — Creazione Plaintext di 32 Byte

```
-rw-r--r--@ 1 nicolabeccaceci  staff  32 Oct  7 09:38 /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI
    INFORMATIVI/LABS/lab02/.tmp_build/ptext
    32 /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI INFORMATIVI/LABS/lab02/.tmp_build/ptext
```

Spiegazione dell'Output: Il comando `wc -c` conferma esattamente 32 byte. Questo dettaglio è essenziale per verificare il comportamento del padding PKCS#7: quando un plaintext è già un multiplo esatto della dimensione del blocco, un cifrario a blocchi standard deve comunque aggiungere un blocco intero di padding (16 byte per AES, portando il totale a 48 byte) per garantire l'univocità della rimozione del padding in fase di decifratura.

2.1.2 Comando 2: Cifratura con AES-128 in Modalità CBC

Alice cifra `ptext` utilizzando AES a 128 bit in modalità CBC. Vengono esplicitati una chiave esadecimale di 32 caratteri (128 bit) e un IV a 16 byte (128 bit).

Alice — Cifratura AES-128-CBC con Stampa Parametri (-p)

```
salt=0000000000000000
key=00112233445566778899AABBCCDDEEFF
iv =0102030405060708090A0B0C0D0E0F10
```

Anatomia dei Parametri del Comando:

- `-e`: Specifica l'operazione di cifratura (*encryption*).
- `-aes-128-cbc`: Seleziona l'Advanced Encryption Standard con chiave a 128 bit e modalità Cipher Block Chaining.
- `-K 001122...`: Passa la chiave simmetrica esplicita in formato esadecimale.
- `-iv 010203...`: Specifica il vettore di inizializzazione a 128 bit, necessario in CBC per rendere non deterministica la cifratura del primo blocco.
- `-p`: Stampa a schermo i valori effettivi di Salt, Chiave e IV caricati nel motore crittografico.

2.1.3 Comandi 3–6: Cifratura con DES, RC4, 3DES e ChaCha20

Alice procede cifrando il medesimo testo con cifrari storici a blocchi (DES a 64 bit, 3DES a tre chiavi EDE) e con cifrari a flusso (RC4 e il moderno ChaCha20).

Alice — Cifratura DES, RC4, 3DES e ChaCha20

```
# Cifratura DES-CBC:
salt=0000000000000000
key=0123456789ABCDEF
iv =123456789ABCDEF0

# Cifratura RC4 (Stream):
salt=0000000000000000
key=00112233445566778899AABBCCDDEEFF

# Cifratura 3DES (DES-EDE3-CBC):
salt=0000000000000000
key=0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF
iv =123456789ABCDEF0

# Cifratura ChaCha20 (Modern Stream Cipher):
hex string is too short, padding with zero bytes to length
salt=0000000000000000
key=00112233445566778899AABBCCDDEEFF000000000000000000000000000000000
iv =0102030405060708090A0B0C0D0E0F10
```

2.1.4 Comando 7: Ispezione Comparativa delle Dimensioni dei File Cifrati

Alice esegue `ls -l` per confrontare le dimensioni risultanti tra il testo in chiaro e tutti i file cifrati.

Alice — Confronto Lunghezza File Cifrati vs Plaintext

```
-rw-r--r--@ 1 nicolabeccaceci  staff  48 Oct  7 09:38 /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI
      INFORMATIVI/LABS/lab02/.tmp_build/ctext.aes128
-rw-r--r--@ 1 nicolabeccaceci  staff  32 Oct  7 09:34 /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI
      INFORMATIVI/LABS/lab02/.tmp_build/ctext.aes128.nopad
-rw-r--r--@ 1 nicolabeccaceci  staff   0 Oct  7 09:34 /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI
      INFORMATIVI/LABS/lab02/.tmp_build/ctext.bad
-rw-r--r--@ 1 nicolabeccaceci  staff  32 Oct  7 09:38 /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI
      INFORMATIVI/LABS/lab02/.tmp_build/ctext.chacha20
-rw-r--r--@ 1 nicolabeccaceci  staff  40 Oct  7 09:38 /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI
      INFORMATIVI/LABS/lab02/.tmp_build/ctext.des
-rw-r--r--@ 1 nicolabeccaceci  staff  40 Oct  7 09:38 /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI
      INFORMATIVI/LABS/lab02/.tmp_build/ctext.des3
-rw-r--r--@ 1 nicolabeccaceci  staff  64 Oct  7 09:34 /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI
      INFORMATIVI/LABS/lab02/.tmp_build/ctext.kdf
-rw-r--r--@ 1 nicolabeccaceci  staff  32 Oct  7 09:38 /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI
      INFORMATIVI/LABS/lab02/.tmp_build/ctext.rc4
-rw-r--r--@ 1 nicolabeccaceci  staff  32 Oct  7 09:38 /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI
      INFORMATIVI/LABS/lab02/.tmp_build/ptext
```

Spiegazione Lineare dei Risultati:

- **ptext (32 byte):** Il messaggio originale non cifrato.

- **ctext.aes128 (48 byte):** La dimensione è aumentata di esattamente 16 byte (32+16 = 48). Poiché AES opera su blocchi da 16 byte e il messaggio era un multiplo esatto, lo standard PKCS#7 ha aggiunto un intero blocco di padding riempito con byte di valore `0x10` (16 in decimale).
- **ctext.des e ctext.des3 (40 byte):** DES e 3DES operano su blocchi da 8 byte. Essendo 32 un multiplo di 8, viene aggiunto un intero blocco di padding da 8 byte ($32 + 8 = 40$).
- **ctext.rc4 e ctext.chacha20 (32 byte):** Trattandosi di **cifrari a flusso (stream ciphers)**, essi non operano su blocchi rigidi ma generano un flusso pseudo-casuale (*keystream*) lungo esattamente quanto il messaggio in chiaro, combinato mediante operazione XOR ($\oplus$). Non vi è alcun padding, pertanto la dimensione in output coincide esattamente con quella in input.

2.1.5 Comando 8: Cifratura AES senza Padding (-nopad) e Fallimento con Dati Disallineati

Alice sperimenta l'opzione `-nopad`. Poiché `ptext` è lungo esattamente 32 byte (multiplo di 16), la cifratura riesce e il file `ctext.aes128.nopad` è lungo esattamente 32 byte. Successivamente Alice tenta di cifrare un messaggio di lunghezza non multipla (`ptext_bad`, 15 byte), ottenendo un errore bloccante.

Alice — Verifica Funzionamento e Fallimento con Flag -nopad

```
# Test riuscito su blocco multiplo di 16 byte:
salt=0000000000000000
key=00112233445566778899AABBCCDDEEFF
iv =0102030405060708090A0B0C0D0E0F10
-rw-r--r--@ 1 nicolabeccaceci  staff  32 Oct  7 09:38 /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI
     INFORMATIVI/LABS/lab02/.tmp_build/ctext.aes128.nopad

# Test fallito su blocco disallineato (15 byte):
bad encrypt
806189F301000000:error:1C80006B:Provider routines:ossl_cipher_generic_block_final:wrong final block length
    :providers/implementations/ciphers/ciphercommon.c:442:
salt=0000000000000000
key=00112233445566778899AABBCCDDEEFF
iv =0102030405060708090A0B0C0D0E0F10
Codice uscita openssl con padding errato: 1
```

2.1.6 Comando 9: Decifratura di Tutti i File e Validazione di Integrità

Bob riceve i file cifrati, applica il flag `-d` (*decrypt*) e verifica tramite `diff -s` che ciascun file decifrato corrisponda bit per bit a `ptext`.

Bob — Decifratura Multipla e Verifica Integrità con diff

```
salt=0000000000000000
key=00112233445566778899AABBCCDDEEFF
iv =0102030405060708090A0B0C0D0E0F10
salt=0000000000000000
key=0123456789ABCDEF
iv =123456789ABCDEF0
salt=0000000000000000
key=00112233445566778899AABBCCDDEEFF
salt=0000000000000000
key=0123456789ABCDEF0123456789ABCDEF0123456789ABCDEF
iv =123456789ABCDEF0
hex string is too short, padding with zero bytes to length
```

Bob — Decifratura Multipla e Verifica Integrità con diff *(continua dalla pagina precedente)*

```
salt=0000000000000000
key=00112233445566778899AABBCCDDEEFF000000000000000000000000000000000
iv =0102030405060708090A0B0C0D0E0F10
Verifica integrità decifratura (diff con ptext):
Files /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI INFORMATIVI/LABS/lab02/.tmp_build/ptext and /
    Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI INFORMATIVI/LABS/lab02/.tmp_build/dtext.aes128 are
     identical
Files /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI INFORMATIVI/LABS/lab02/.tmp_build/ptext and /
    Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI INFORMATIVI/LABS/lab02/.tmp_build/dtext.des are
    identical
Files /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI INFORMATIVI/LABS/lab02/.tmp_build/ptext and /
    Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI INFORMATIVI/LABS/lab02/.tmp_build/dtext.rc4 are
    identical
Files /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI INFORMATIVI/LABS/lab02/.tmp_build/ptext and /
    Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI INFORMATIVI/LABS/lab02/.tmp_build/dtext.des3 are
    identical
Files /Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI INFORMATIVI/LABS/lab02/.tmp_build/ptext and /
    Users/nicolabeccaceci/Desktop/SICUREZZA DEI SISTEMI INFORMATIVI/LABS/lab02/.tmp_build/dtext.chacha20
    are identical
```

2.1.7 Comando 10: Derivazione di Chiave e IV tramite Passphrase (KDF)

Cosa accade quando non si specificano manualmente i parametri `-K` e `-iv`? OpenSSL attiva una Key Derivation Function (storicamente `EVP_BytesToKey`) che genera un salt crittografico casuale di 8 byte e deriva chiave e IV tramite hashing iterativo.

Alice — Cifratura Basata su Passphrase con Salt Casuale

```
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
salt=F4A51A83DD8847B1
key=45ACF81095749886D68D92265F32F387
iv =E2D327363D6905E36BCDEF942C76C513
```

Spiegazione dell'Output: In questo caso viene generato un `salt` non nullo. Nei primi byte del file cifrato OpenSSL scriverà la stringa di intestazione `Salted__` seguita dagli 8 byte del salt.

Attenzione alle Trappole & Errori Frequenti

Differenza Cruciale tra -k (minuscolo) e -K (maiuscolo):

- `-k passphrase`: Accetta una stringa di testo arbitraria che viene processata dalla funzione KDF per derivare matematicamente la vera chiave.
- `-K chiave_hex`: Richiede la chiave crittografica effettiva espressa come sequenza di cifre esadecimali, senza alcun passaggio attraverso funzioni KDF. Se la sequenza è più corta del necessario, OpenSSL aggiunge zeri in coda; se è più lunga, tronca i byte eccedenti.

2.2 Esercizio 1.2: Attacco a Forza Bruta su Cifrario a Flusso (RC4)

In questo esercizio Alice cifra un messaggio riservato tramite RC4 impostando una chiave brevissima, costituita da un singolo carattere esadecimale (4 bit, ovvero soltanto 16 combinazioni possibili: da `0` ad `F`).



Figura 2: Dinamica dell'Attacco a Forza Bruta su Spazio di Chiavi Ridotto (4 bit).

2.2.1 Comando 1: Esecuzione dello Script di Brute-Force su Messaggio di Testo

Mallory itera sulle 16 chiavi possibili eseguendo la decifratura e verificando se l'output contiene esclusivamente caratteri stampabili ASCII.

Mallory — Cracking a Forza Bruta su alice_ctext (Spazio a 4-bit)

```
[GENERATORE ALICE] Generato alice_ctext con chiave casuale a 4-bit (chiave reale segreta: F)
-rw-r--r--  1 nicolabeccaceci  staff  49 Oct  7 09:38 alice_ctext
[BRUTE FORCE] Tentativo di cracking chiave a 4 bit su alice_ctext...
Iterazione su spazio esadecimale (16 tentativi: 0, 1, 2, ..., E, F):
Provando chiave 0 -> scartata (byte non stampabili)
Provando chiave 1 -> scartata (byte non stampabili)
...
Provando chiave E -> scartata (byte non stampabili)
Provando chiave F -> [SUCCESSO] Chiave identificata: F
[CONTENUTO DECIFRATO]: Questo e' un messaggio segreto di Alice per Bob!
```

Spiegazione del Successo: Poiché il messaggio originale è redatto in lingua italiana (caratteri ASCII standard), solo la chiave corretta produce una sequenza interamente stampabile priva di byte di controllo non validi.

2.2.2 Comando 2: Tentativo di Brute-Force su File Binario Casuale (bruteforce_enc)

Mallory tenta di ripetere la medesima procedura su un file che in origine conteneva dati pseudo-casuali (`bruteforce_enc`).

Mallory — Tentativo di Decifratura su File Pseudo-Casuale

```
[TEST SU FILE PSEUDO-RANDOM] Esecuzione attacco su bruteforce_enc:
Key 0:          a1  8c  19  c5  b7  b7  e5  f7  5a  2f  0e  7b  db  f7  85  1d
Key 1:          15  b3  64  c4  76  1d  c4  61  e0  66  e4  d8  2e  a2  fc  38
Key 2:          60  cc  fc  2b  72  e8  e0  65  7f  5f  f6  cb  47  e6  68  56
Key 3:          55  ba  03  20  bc  60  7c  43  03  ee  85  c3  19  b4  2d  39
Key 4:          84  c1  2d  ad  97  c0  db  cf  98  a7  8b  fe  61  1f  01  56
Key 5:          69  0d  86  18  94  ef  ff  fd  98  69  7b  1b  2e  fc  6e  47
Key 6:          fb  14  fd  31  50  b9  f4  59  b6  04  3b  82  04  8b  a7  8b
Key 7:          15  06  24  3e  e1  21  ce  18  f1  84  12  63  04  e0  e9  12
Key 8:          35  28  ec  b5  79  d2  5b  32  dd  de  27  19  b5  72  6c  a3
```

Mallory — Tentativo di Decifratura su File Pseudo-Casuale *(continua dalla pagina precedente)*

```
Key 9:          f9  d8  9f  85  9b  8f  96  a2  b2  ae  8e  89  f2  60  c5  d7
Key A:          bc  e2  72  61  f3  66  b8  d7  19  e3  b6  09  c7  96  7b  e7
Key B:          53  c5  35  3b  78  35  10  fd  49  d2  e0  0e  c2  32  b4  7c
Key C:          be  e6  a3  dc  3d  86  6a  bf  d6  36  96  9c  ca  81  70  1e
Key D:          6a  f1  f3  17  62  3e  73  4e  f2  56  7e  85  69  2f  c1  ad
Key E:          04  d8  7d  4f  60  44  e7  dc  e1  5b  46  52  69  c1  cc  90
Key F:          3a  6f  e2  2f  dd  68  ad  c0  43  5e  17  0d  b2  4f  fc  fd
Conclusione: Impossibile distinguere il chiaro senza ridondanza nota a priori (Unicity Distance).
```

Analisi Crittanalitica e Unicity Distance: L'attacco a forza bruta fallisce completamente perché l'attaccante **non dispone di un criterio di decisione (oracolo)** per stabilire quale delle 16 sequenze decifrate sia quella autentica. Essendo il chiaro originale una sequenza di byte randomici generati da un generatore crittografico, ogni chiave esadecimale restituisce un flusso altrettanto casuale e plausibile. Questo fenomeno si ricollega al concetto di **Unicity Distance di Shannon**:

$$U = \frac{H(K)}{D} \tag{1}$$

dove $H(K)$ è l'entropia della chiave e D è la ridondanza della sorgente di testo. Quando la ridondanza è nulla ($D = 0$), come nel caso di file compressi, cifrati o puramente casuali, l'unicity distance tende a infinito e la crittanalisi a forza bruta non può determinare la chiave corretta neppure provando l'intero spazio delle chiavi.

2.3 Esercizio 1.3: Benchmark e Valutazione delle Prestazioni Simmetriche

Si valutano le prestazioni comparative dei cifrari simmetrici variando le dimensioni del file da 100 B fino a 10 MB.

Host — Benchmark Prestazioni con time e File di Dimensione Variabile

```
--- Benchmark file: r_100B.dat ---
AES-128-CBC: real       0m0.007s
AES-256-CBC: real       0m0.007s
ChaCha20:    real       0m0.007s
--- Benchmark file: r_10KB.dat ---
AES-128-CBC: real       0m0.007s
AES-256-CBC: real       0m0.007s
ChaCha20:    real       0m0.007s
--- Benchmark file: r_1MB.dat ---
AES-128-CBC: real       0m0.008s
AES-256-CBC: real       0m0.008s
ChaCha20:    real       0m0.008s
--- Benchmark file: r_10MB.dat ---
AES-128-CBC: real       0m0.019s
AES-256-CBC: real       0m0.021s
ChaCha20:    real       0m0.016s
```

Successivamente si esegue il test standard di throughput della libreria tramite `openssl speed aes-128-cbc`.

Host — Throughput Hardware AES-128-CBC (openssl speed)

```
Doing aes-128-cbc ops for 1s on 16 size blocks: 49079743 aes-128-cbc ops in 1.00s
Doing aes-128-cbc ops for 1s on 64 size blocks: 20951816 aes-128-cbc ops in 0.99s
Doing aes-128-cbc ops for 1s on 256 size blocks: 6093444 aes-128-cbc ops in 1.00s
Doing aes-128-cbc ops for 1s on 1024 size blocks: 1561072 aes-128-cbc ops in 0.99s
Doing aes-128-cbc ops for 1s on 8192 size blocks: 198953 aes-128-cbc ops in 0.99s
Doing aes-128-cbc ops for 1s on 16384 size blocks: 99454 aes-128-cbc ops in 1.00s
version: 3.5.7
built on: Tue Jun  9 19:50:21 2026 UTC
options: bn(64,64)
compiler: arm64-apple-darwin20.0.0-clang -fPIC -arch arm64 -ftree-vectorize -fPIC -fstack-protector-strong
      -O2 -pipe -isystem /Users/nicolabeccaceci/miniconda3/include -fdebug-prefix-map=/Users/ec2-user/croot
     /openssl_1781034588958/work=/usr/local/src/conda/openssl-3.5.7 -fdebug-prefix-map=/Users/
     nicolabeccaceci/miniconda3=/usr/local/src/conda-prefix -DL_ENDIAN -DOPENSSL_PIC -D_REENTRANT -
     DOPENSSL_BUILDING_OPENSSL -DNDEBUG -D_FORTIFY_SOURCE=2 -DNDEBUG -isystem /Users/nicolabeccaceci/
     miniconda3/include -mmacosx-version-min=12.1  -mmacosx-version-min=12.1
CPUINFO: OPENSSL_armcap=0x87d
The 'numbers' are in 1000s of bytes per second processed.
type             16 bytes     64 bytes    256 bytes   1024 bytes   8192 bytes  16384 bytes
aes-128-cbc     785275.89k  1354460.83k  1559921.66k  1614684.57k  1646285.83k  1629454.34k

################################################################################
[ESERCIZIO 2: ALGORITMI DI CIFRATURA ASIMMETRICA]
################################################################################
```

Interpretazione del Comando time vs speed:

- Nel comando `time`, il parametro rilevante per il costo computazionale dell'algoritmo è **user time** (tempo speso dalla CPU in modalità utente), mentre **sys** misura il costo delle chiamate di sistema I/O del kernel e **real** il tempo di orologio da parete (*wall-clock*).

- Il comando `speed` è molto più accurato per blocchi di piccole dimensioni perché ripete l'operazione milioni di volte in un secondo, neutralizzando i costi fissi di avvio del processo (*overhead* di invocazione della shell).

3 Esercizio 2: Algoritmi di Cifratura Asimmetrica, RSA ed Curve Ellittiche

3.1 Esercizio 2.1: Generazione di Chiavi RSA ed Ispezione Parametri PKCS#1

Alice genera una coppia di chiavi RSA a 2048 bit memorizzandola nel file `rsa.key.alice`.

Alice — Generazione Chiave RSA 2048-bit

```
-rw-------@ 1 nicolabeccaceci  staff  1704 Oct  7 09:38 rsa.key.alice
```

Alice ispeziona la struttura interna della chiave privata tramite il comando `openssl rsa -text`.

Alice — Ispezione Parametri Interni PKCS#1 e Teorema Cinese del Resto

```
Private-Key: (2048 bit, 2 primes)
modulus:
    00:df:88:6f:72:1c:87:b8:26:8a:f7:55:7d:30:6d:
    0f:db:a3:8a:43:5a:52:85:19:12:7b:96:2a:c8:0d:
    a7:b0:ea:02:3d:a0:4f:e4:34:48:d2:d4:33:98:92:
    ab:b7:36:ff:ed:4f:c6:f7:fc:4a:94:a5:ad:90:46:
    1b:77:10:23:94:38:47:ff:d6:e7:f2:5c:42:ec:d7:
    47:b8:83:ba:7d:24:5a:b9:c7:de:22:92:16:33:f0:
    50:12:99:41:7f:29:01:68:71:09:e9:f2:17:82:8b:
    ee:a4:38:19:28:13:48:9e:ac:6d:5d:0d:83:49:67:
    0b:53:b9:c7:ee:24:31:eb:69:b4:60:f2:27:d7:8d:
    37:bd:11:ce:40:cc:a3:e9:f2:e2:8f:17:33:ff:d2:
    0e:8c:72:1d:01:b2:58:7a:f2:8d:ad:12:0c:2a:54:
    5a:c1:72:ad:a2:69:4f:5f:3f:ec:4c:a4:3f:a0:18:
    32:0f:6b:94:7d:29:fd:57:4c:6d:e3:46:d4:bf:e2:
    bf:67:71:29:6c:e1:e6:49:07:12:09:aa:28:c3:e4:
    39:b2:44:6a:07:5e:d1:6d:7b:aa:b9:f2:1b:11:7f:
    6a:25:b0:8e:cd:da:d9:3f:e6:8b:fe:40:00:6c:d5:
    32:6b:85:5b:2b:fe:2a:85:14:81:93:47:b1:b1:e4:
    a7:6d
publicExponent: 65537 (0x10001)
privateExponent:
    5a:01:01:7e:9c:e6:38:cb:ac:20:b2:97:59:ac:50:

    3a:4b:bb:1c:2b:7e:16:81:65:e3:c5:2f:2c:a5:f2:
    e3:55:15:d9:e2:76:c8:98:14:ca:6f:0e:e7:5c:07:
    30:1a:41:2a:38:79:10:d8:d0:de:d1:66:f5:34:85:
    52:87:79:0b:f1:2d:6c:62:07:ba:9a:53:88:31:96:
... [output troncato: ulteriori 8 righe consultabili in log.txt]
```

Decomposizione dei Parametri Matematici RSA:

- **modulus (n):** Prodotto dei due grandi numeri primi $n = p \times q$. È il modulo delle operazioni aritmetiche, di lunghezza pari a 2048 bit. Fa parte sia della chiave pubblica che privata.
- **publicExponent (e):** Fissato di default da OpenSSL a **65537** ($2^{16}+1$, noto come F_4, quarto numero primo di Fermat). La sua rappresentazione binaria contiene solo due '1' (`0b10000000000000001`), consentendo un'esponenziazione modulare velocissima con solo 17 moltiplicazioni e un quadrato.
- **privateExponent (d):** L'inverso moltiplicativo di e modulo $\phi(n)$: $d \equiv e^{-1} \pmod{(p-1)(q-1)}$.
- **prime1 (p) e prime2 (q):** I fattori primi segreti di n.

- **exponent1** (dP), **exponent2** (dQ) **e coefficient** ($qInv$)**:** Parametri precalcolati per velocizzare le operazioni di firma e decifratura privata tramite il **Chinese Remainder Theorem (CRT)**, accelerando il calcolo di un fattore $\approx 4\times$.

Alice estrae la sola componente pubblica nel file `rsa.pubkey.alice`:

Alice — Estrazione e Visualizzazione Chiave Pubblica RSA

```
writing RSA key
Public-Key: (2048 bit)
Modulus:
    00:df:88:6f:72:1c:87:b8:26:8a:f7:55:7d:30:6d:
    0f:db:a3:8a:43:5a:52:85:19:12:7b:96:2a:c8:0d:
    a7:b0:ea:02:3d:a0:4f:e4:34:48:d2:d4:33:98:92:
    ab:b7:36:ff:ed:4f:c6:f7:fc:4a:94:a5:ad:90:46:
    1b:77:10:23:94:38:47:ff:d6:e7:f2:5c:42:ec:d7:
    47:b8:83:ba:7d:24:5a:b9:c7:de:22:92:16:33:f0:
    50:12:99:41:7f:29:01:68:71:09:e9:f2:17:82:8b:
    ee:a4:38:19:28:13:48:9e:ac:6d:5d:0d:83:49:67:
    0b:53:b9:c7:ee:24:31:eb:69:b4:60:f2:27:d7:8d:
    37:bd:11:ce:40:cc:a3:e9:f2:e2:8f:17:33:ff:d2:
    0e:8c:72:1d:01:b2:58:7a:f2:8d:ad:12:0c:2a:54:
    5a:c1:72:ad:a2:69:4f:5f:3f:ec:4c:a4:3f:a0:18:
    32:0f:6b:94:7d:29:fd:57:4c:6d:e3:46:d4:bf:e2:
    bf:67:71:29:6c:e1:e6:49:07:12:09:aa:28:c3:e4:
    39:b2:44:6a:07:5e:d1:6d:7b:aa:b9:f2:1b:11:7f:
    6a:25:b0:8e:cd:da:d9:3f:e6:8b:fe:40:00:6c:d5:
    32:6b:85:5b:2b:fe:2a:85:14:81:93:47:b1:b1:e4:
    a7:6d
Exponent: 65537 (0x10001)
```

3.2 Esercizio 2.2: Cifratura e Decifratura Asimmetrica con pkeyutl e Limiti Dimensionali

Per trasmettere un segreto confidenziale ad Alice, Bob deve cifrare con la **chiave pubblica di Alice**. In tal modo solo Alice, custode esclusiva della chiave privata complementare, potrà decifrare il payload.

Bob/Alice — Cifratura e Decifratura con pkeyutl

```
-rw-r--r--@ 1 nicolabeccaceci  staff  33 Oct  7 09:38 plain
Cifratura asimmetrica per Alice (con la sua chiave pubblica):
-rw-r--r--@ 1 nicolabeccaceci  staff  256 Oct  7 09:38 encRSA
Decifratura asimmetrica da parte di Alice (con la sua chiave privata):
Un grandissimo segreto per Alice
Files plain and decRSA are identical
```

3.2.1 Dimostrazione del Limite Dimensionale di RSA

Cosa succede se si tenta di cifrare un file di grandi dimensioni (ad esempio un PDF o un file da 10 KB) direttamente con RSA?

Alice — Tentativo di Cifratura File Grande con RSA (Fallimento per Limite Modulo)

```
Public Key operation error
806189F301000000:error:0200006E:rsa routines:ossl_rsa_padding_add_PKCS1_type_2_ex:data too large for key
    size:crypto/rsa/rsa_pk1.c:132:
Codice uscita cifratura file grande con RSA (fallimento per superamento modulo): 1
```

Fondamento Teorico: Matematicamente, RSA opera su interi modulo n, quindi il dato in chiaro deve essere strettamente minore di n ($m < n$). Con lo schema di padding standard PKCS#1 v1.5, per prevenire attacchi di malleabilità è obbligatorio inserire almeno 11 byte di padding strutturato:

$$\text{Dimensione Massima Plaintext} = N_{\text{chiave}} - 11 \text{ byte} \tag{2}$$

Per una chiave a 2048 bit (256 byte), la dimensione massima del dato cifrabile direttamente è di appena:

$$256 - 11 = \mathbf{245} \text{ byte}$$

Per file superiori a 245 byte la cifratura fallisce. Nella pratica industriale si adotta sempre uno **schema ibrido** (es. envelope digitale): si cifra il documento voluminoso con un cifrario simmetrico rapido (AES-256) e si cifra con RSA la sola chiave simmetrica di sessione (32 byte).

3.3 Esercizio 2.3: Firma Digitale RSA e Meccanismo di Verifica

A differenza della cifratura (in cui chiunque può cifrare con la chiave pubblica e solo il destinatario decifra con la privata), nella **firma digitale**:

- Il firmatario usa la propria **chiave privata** per generare la firma.
- Chiunque conosca la corrispondente **chiave pubblica** può verificare l'autenticità e l'integrità del dato.

Alice — Firma Digitale RSA e Doppia Verifica (verifyrecover e verify)

```
Creazione firma digitale su 'plain' con chiave privata di Alice:
-rw-r--r--@ 1 nicolabeccaceci  staff  256 Oct  7 09:38 sig.alice
Verifica e Recupero (verifyrecover) con chiave pubblica di Alice:
Un grandissimo segreto per Alice
Verifica formale della firma (verify) con chiave pubblica di Alice:
Signature Verified Successfully
Test verifica firma con payload alterato (fallimento atteso):
806189F301000000:error:030000EA:digital envelope routines:EVP_PKEY_verify:provider signature failure:
    crypto/evp/signature.c:1176:RSA verify:
Signature Verification Failure
Codice uscita verifica firma manomessa: 1
```

Spiegazione dei Flag:

- `-sign`: Esegue l'esponenziazione privata $S = m^d \pmod{n}$.
- `-verifyrecover`: Applica la chiave pubblica alla firma ($S^e \pmod{n}$) ricostruendo direttamente il plaintext originale.
- `-verify`: Confronta matematicamente il digest del file passato in input con la firma salvata in `-sigfile`. Se un intruso altera anche un singolo carattere del messaggio, l'operazione restituisce fallimento immediato.

3.4 Esercizi 2.4 & 2.5: Crittografia su Curve Ellittiche (EC) e Firma ECDSA

Alice genera una chiave su curva ellittica SECG su campo primo a 192 bit (`secp192k1`) ed esegue una firma digitale con l'algoritmo ECDSA.

Alice — Generazione Parametri EC ed Estrazione Chiave Pubblica

```
secp192k1 : SECG curve over a 192 bit prime field
-rw-------@ 1 nicolabeccaceci  staff  261 Oct  7 09:38 ec.key.alice
Estrazione chiave pubblica EC di Alice:
read EC key
writing EC key
read EC key
Public-Key: (192 bit)
pub:
    04:0f:e9:90:a4:95:fa:e3:70:36:f4:fb:99:80:73:
    92:f2:1d:b9:27:6e:b9:6e:f8:61:b7:dc:a8:a8:35:
    d7:2e:40:cf:4e:1c:da:04:5b:41:e9:d8:c0:22:05:
    22:73:bf:73
ASN1 OID: secp192k1
```

Successivamente Alice firma il file `plain` con ECDSA e ne verifica la validità:

Alice — Firma e Verifica Digitale ECDSA

```
Firma digitale con ECDSA su 'plain':
-rw-r--r--@ 1 nicolabeccaceci  staff  56 Oct  7 09:38 ecsig.alice
Verifica formale ECDSA con chiave pubblica EC:
Signature Verified Successfully
```

3.5 Esercizio 2.6: Benchmark Comparativo delle Prestazioni Asimmetriche

Si eseguono test approfonditi di throughput asimmetrico confrontando le diverse lunghezze di chiave e le famiglie di algoritmi:

Host — Benchmark Comparativo openssl speed (RSA, DSA, ECDSA, AES)

```
--- Speed RSA (512 vs 1024 vs 2048 vs 4096 bit) ---
Doing 512 bits private rsa sign ops for 1s: 43849 512 bits private RSA sign ops in 1.00s
Doing 512 bits public rsa verify ops for 1s: 558241 512 bits public RSA verify ops in 0.99s
Doing 512 bits public rsa encrypt ops for 1s: 482641 512 bits public RSA encrypt ops in 1.00s
Doing 512 bits private rsa decrypt ops for 1s: 36907 512 bits private RSA decrypt ops in 0.98s
Doing 1024 bits private rsa sign ops for 1s: 9847 1024 bits private RSA sign ops in 1.00s
Doing 1024 bits public rsa verify ops for 1s: 213112 1024 bits public RSA verify ops in 1.00s
Doing 1024 bits public rsa encrypt ops for 1s: 175674 1024 bits public RSA encrypt ops in 0.92s
Doing 1024 bits private rsa decrypt ops for 1s: 7530 1024 bits private RSA decrypt ops in 0.86s
Doing 2048 bits private rsa sign ops for 1s: 682 2048 bits private RSA sign ops in 0.54s
Doing 2048 bits public rsa verify ops for 1s: 45669 2048 bits public RSA verify ops in 0.84s
Doing 2048 bits public rsa encrypt ops for 1s: 49043 2048 bits public RSA encrypt ops in 0.88s
Doing 2048 bits private rsa decrypt ops for 1s: 1604 2048 bits private RSA decrypt ops in 1.00s
Doing 4096 bits private rsa sign ops for 1s: 259 4096 bits private RSA sign ops in 1.00s
Doing 4096 bits public rsa verify ops for 1s: 17588 4096 bits public RSA verify ops in 1.00s
Doing 4096 bits public rsa encrypt ops for 1s: 16616 4096 bits public RSA encrypt ops in 0.97s
Doing 4096 bits private rsa decrypt ops for 1s: 247 4096 bits private RSA decrypt ops in 0.98s
Doing rsa512 keygen ops for 1s: 471 rsa512 KEM keygen ops in 0.89s
Doing rsa512 encaps ops for 1s: 366160 rsa512 KEM encaps ops in 0.94s
Doing rsa512 decaps ops for 1s: 41177 rsa512 KEM decaps ops in 0.95s
Doing rsa1024 keygen ops for 1s: 99 rsa1024 KEM keygen ops in 0.99s
Doing rsa1024 encaps ops for 1s: 177386 rsa1024 KEM encaps ops in 1.00s
Doing rsa1024 decaps ops for 1s: 9804 rsa1024 KEM decaps ops in 1.00s
Doing rsa2048 keygen ops for 1s: 23 rsa2048 KEM keygen ops in 1.01s
Doing rsa2048 encaps ops for 1s: 58554 rsa2048 KEM encaps ops in 0.99s
Doing rsa2048 decaps ops for 1s: 1585 rsa2048 KEM decaps ops in 0.99s
Doing rsa4096 keygen ops for 1s: 2 rsa4096 KEM keygen ops in 1.54s
Doing rsa4096 encaps ops for 1s: 16913 rsa4096 KEM encaps ops in 1.00s
Doing rsa4096 decaps ops for 1s: 249 rsa4096 KEM decaps ops in 0.98s
Doing rsa512 keygen ops for 1s: 495 rsa512 signature keygen ops in 0.96s
Doing rsa512 signs ops for 1s: 43396 rsa512 signature sign ops in 0.99s
Doing rsa512 verify ops for 1s: 555105 rsa512 signature verify ops in 0.99s
Doing rsa1024 keygen ops for 1s: 101 rsa1024 signature keygen ops in 0.98s
Doing rsa1024 signs ops for 1s: 9734 rsa1024 signature sign ops in 0.99s
Doing rsa1024 verify ops for 1s: 206651 rsa1024 signature verify ops in 0.99s
Doing rsa2048 keygen ops for 1s: 16 rsa2048 signature keygen ops in 1.02s
Doing rsa2048 signs ops for 1s: 1627 rsa2048 signature sign ops in 1.01s
Doing rsa2048 verify ops for 1s: 64724 rsa2048 signature verify ops in 0.99s
Doing rsa4096 keygen ops for 1s: 2 rsa4096 signature keygen ops in 1.40s
Doing rsa4096 signs ops for 1s: 252 rsa4096 signature sign ops in 0.98s
Doing rsa4096 verify ops for 1s: 17694 rsa4096 signature verify ops in 1.00s
version: 3.5.7
built on: Tue Jun  9 19:50:21 2026 UTC
options: bn(64,64)
compiler: arm64-apple-darwin20.0.0-clang -fPIC -arch arm64 -ftree-vectorize -fPIC -fstack-protector-strong
      -O2 -pipe -isystem /Users/nicolabeccaceci/miniconda3/include -fdebug-prefix-map=/Users/ec2-user/croot
     /openssl_1781034588958/work=/usr/local/src/conda/openssl-3.5.7 -fdebug-prefix-map=/Users/
     nicolabeccaceci/miniconda3=/usr/local/src/conda-prefix -DL_ENDIAN -DOPENSSL_PIC -D_REENTRANT -
     DOPENSSL_BUILDING_OPENSSL -DNDEBUG -D_FORTIFY_SOURCE=2 -DNDEBUG -isystem /Users/nicolabeccaceci/
     miniconda3/include -mmacosx-version-min=12.1  -mmacosx-version-min=12.1
... [output troncato: ulteriori 126 righe consultabili in log.txt]
```

Analisi delle Prestazioni e Considerazioni di Progetto:

1. **Comportamento asimmetrico di RSA (Firma vs Verifica):** Le operazioni con chiave pubblica (*verify* ed *encrypt*) sono ordini di grandezza più veloci delle operazioni con chiave privata (*sign* e *decrypt*). Ciò è dovuto al fatto che l'esponente pubblico $e = 65537$ ha un peso di Hamming piccolissimo (17 moltiplicazioni modulari), mentre l'esponente privato d è un numero intero di 2048 bit casuale, richiedendo centinaia di esponenziazioni pesanti.

2. **Crescita della dimensione di chiave RSA:** Passando da 512 bit a 4096 bit, le prestazioni di firma crollano di oltre 150 volte, poiché il costo computazionale scala come $O(k^3)$ rispetto alla lunghezza k della chiave.

3. **Curve Ellittiche (ECDSA) vs RSA:** Le curve ellittiche garantiscono livelli di sicurezza equivalenti con chiavi drammaticamente più corte (es. una curva a 256 bit offre sicurezza paragonabile a RSA-3072). Inoltre in ECDSA l'operazione di firma è sensibilmente più rapida rispetto alla verifica.

4. **Rapporto Simmetrico vs Asimmetrico:** Cifrare con AES-128 consente un throughput di svariati gigabyte al secondo (grazie anche alle istruzioni hardware dedicate AES-NI o ARM Crypto), mentre cifrare con RSA-2048 elabora poche centinaia di kilobyte al secondo. La crittografia simmetrica è mediamente da **1000 a 10.000 volte più veloce** di quella asimmetrica.

4 Esercizio 3: Algoritmi di Digest, Effetto Valanga e Collisioni

4.1 Esercizio 3.1: Calcolo Digest, Effetto Valanga e Costruzione CBC-MAC

Alice calcola l'impronta crittografica del messaggio `msg` utilizzando quattro algoritmi di generazioni diverse: MD5, SHA-1, SHA2-256 e SHA3-256.

Alice — Calcolo Digest di Prova (MD5, SHA1, SHA256, SHA3)

```
-rw-r--r--@ 1 nicolabeccaceci  staff  58 Oct  7 09:39 msg
Valori digest calcolati per 'msg':
MD5(msg)= 11b3179f61fd6dacb5a881454bd6ef12
SHA1(msg)= 4b0b732d0c658d79d1d2fa8e84dffd5f2f8ed896
SHA2-256(msg)= 236a12af73b8a2072259a9e029ced9f468b5d7a478d7b24b1859e284ba5ad2c0
SHA3-256(msg)= 6d84f96b413d6e576602709a976c3b73b7923a3dd8685aa6b172452a50c52689
```

4.1.1 Dimostrazione dell'Effetto Valanga (Avalanche Effect)

Alice rimuove un singolo carattere (il punto esclamativo finale) dal messaggio e ricalcola il digest SHA-256:

Alice — Dimostrazione Empirica dell'Effetto Valanga in SHA-256

```
SHA-256 originale:  SHA2-256(msg)= 236a12af73b8a2072259a9e029ced9f468b5d7a478d7b24b1859e284ba5ad2c0
SHA-256 modificato: SHA2-256(msg_modified)=
    c1ae84073fb4b5d02bfacec6b6820c553d8e04aa3019bf2c865efc8b005dcef2
```

Analisi Crittografica: Benché i due messaggi differiscano per un solo byte, l'output risultante è completamente scorrelato: oltre il 50% dei singoli bit dell'hash cambia valore. Questa proprietà fondamentale garantisce che dall'analisi del digest non sia possibile inferire alcuna vicinanza o correlazione con il testo in chiaro originale.

4.1.2 Costruzione di una Funzione Hash con Cifrario a Blocchi (CBC-MAC)

Come evidenziato dalle soluzioni ufficiali, è possibile utilizzare un cifrario a blocchi in modalità CBC con chiave fissa e $IV = 0$ interpretando l'ultimo blocco cifrato come il digest del messaggio:

Alice — Costruzione Digest Basata su Cifrario a Blocchi (CBC-MAC)

```
Ultimo blocco a 16 byte (CBC-MAC digest):
          c2  af  9b  0e  99  70  9e  a6  78  d6  2b  50  c2  f6  8a  bc
```

4.2 Esercizio 3.2: Analisi delle Collisioni con Hash Ridotto a 4-bit (IHA)

Si analizza la robustezza delle funzioni di hash simulando un algoritmo debole a causa della lunghezza ridotta del digest: `IHA` produce un hash di soli 4 bit ($2^4 = 16$ stati possibili).

Alice — Ricerca di Collisioni con IHA (Insecure Hash Algorithm)

```
IHA: F
Target IHA hash per 'msg': F
Collisione trovata con successo dopo 2 tentativi (aggiunti 2 spazi in coda al file).
```

Alice — Ricerca di Collisioni con IHA (Insecure Hash Algorithm) *(continua dalla pagina precedente)*

```
Digest finale di msg1: F
```

Distinzione Teorica tra Pre-Image e Collision Attack:

- **Prima Pre-Immagine (Target Fisso):** Dato un hash fissato $H(m)$, trovare un messaggio m' tale che $H(m') = H(m)$. Richiede in media 2^n tentativi ($2^4 = 16$ prove con IHA).
- **Seconda Pre-Immagine / Collisione Arbitraria (Paradosso del Compleanno):** Trovare *due messaggi qualsiasi* $m_1 \neq m_2$ tali che $H(m_1) = H(m_2)$. Grazie al paradosso del compleanno, il numero di tentativi necessari scende a:

$$2^{n/2} = 2^{4/2} = 2^2 = \mathbf{4} \text{ tentativi} \tag{3}$$

  Per garantire una resistenza di 128 bit alle collisioni arbitrarie, un algoritmo di hash moderno deve produrre un digest di lunghezza almeno doppia: $n = 256$ bit (come in SHA-256).

4.3 Esercizio 3.3: Benchmark degli Algoritmi di Hash

Host — Benchmark Throughput Funzioni Hash (openssl speed)

```
Doing md5 ops for 1s on 16 size blocks: 5167406 md5 ops in 1.00s
Doing md5 ops for 1s on 64 size blocks: 3144668 md5 ops in 1.00s
Doing md5 ops for 1s on 256 size blocks: 1410658 md5 ops in 0.98s
Doing md5 ops for 1s on 1024 size blocks: 439424 md5 ops in 1.00s
Doing md5 ops for 1s on 8192 size blocks: 58980 md5 ops in 1.00s
Doing md5 ops for 1s on 16384 size blocks: 29662 md5 ops in 1.00s
Doing sha1 ops for 1s on 16 size blocks: 8514675 sha1 ops in 0.99s
Doing sha1 ops for 1s on 64 size blocks: 7263851 sha1 ops in 0.99s
Doing sha1 ops for 1s on 256 size blocks: 4058702 sha1 ops in 1.00s
Doing sha1 ops for 1s on 1024 size blocks: 1438454 sha1 ops in 1.00s
Doing sha1 ops for 1s on 8192 size blocks: 205148 sha1 ops in 1.00s
Doing sha1 ops for 1s on 16384 size blocks: 103475 sha1 ops in 1.00s
Doing sha256 ops for 1s on 16 size blocks: 8874101 sha256 ops in 0.99s
Doing sha256 ops for 1s on 64 size blocks: 7327650 sha256 ops in 1.00s
Doing sha256 ops for 1s on 256 size blocks: 3983680 sha256 ops in 0.99s
Doing sha256 ops for 1s on 1024 size blocks: 1436180 sha256 ops in 0.99s
Doing sha256 ops for 1s on 8192 size blocks: 203259 sha256 ops in 1.00s
Doing sha256 ops for 1s on 16384 size blocks: 101351 sha256 ops in 0.97s
Doing sha512 ops for 1s on 16 size blocks: 4027376 sha512 ops in 0.82s
Doing sha512 ops for 1s on 64 size blocks: 4498682 sha512 ops in 0.92s
Doing sha512 ops for 1s on 256 size blocks: 1992793 sha512 ops in 0.92s
Doing sha512 ops for 1s on 1024 size blocks: 754516 sha512 ops in 0.94s
Doing sha512 ops for 1s on 8192 size blocks: 108768 sha512 ops in 0.89s
Doing sha512 ops for 1s on 16384 size blocks: 56573 sha512 ops in 0.93s
version: 3.5.7
built on: Tue Jun  9 19:50:21 2026 UTC
options: bn(64,64)
compiler: arm64-apple-darwin20.0.0-clang -fPIC -arch arm64 -ftree-vectorize -fPIC -fstack-protector-strong
      -O2 -pipe -isystem /Users/nicolabeccaceci/miniconda3/include -fdebug-prefix-map=/Users/ec2-user/croot
     /openssl_1781034588958/work=/usr/local/src/conda/openssl-3.5.7 -fdebug-prefix-map=/Users/
     nicolabeccaceci/miniconda3=/usr/local/src/conda-prefix -DL_ENDIAN -DOPENSSL_PIC -D_REENTRANT -
     DOPENSSL_BUILDING_OPENSSL -DNDEBUG -D_FORTIFY_SOURCE=2 -DNDEBUG -isystem /Users/nicolabeccaceci/
     miniconda3/include -mmacosx-version-min=12.1  -mmacosx-version-min=12.1
CPUINFO: OPENSSL_armcap=0x87d
The 'numbers' are in 1000s of bytes per second processed.
type             16 bytes     64 bytes    256 bytes   1024 bytes   8192 bytes  16384 bytes
md5              82678.50k   201258.75k   368498.42k   449970.18k   483164.16k   485982.21k
sha1            137610.91k   469582.29k  1039027.71k  1472976.90k  1680572.42k  1695334.40k
```

Host — Benchmark Throughput Funzioni Hash (openssl speed) ***(continua dalla pagina precedente)***

```
sha256          143419.81k   468969.60k  1030123.31k  1485503.35k  1665097.73k  1711891.53k
sha512           78582.95k   312951.79k   554516.31k   821940.83k  1001154.44k   996658.10k
... [output troncato: ulteriori 4 righe consultabili in log.txt]
```

5 Esercizio 4: Approfondimenti ed Esercizi Opzionali

5.1 Esercizio 4.1: Manomissione Bit-Flipping su Cifrario a Flusso vs a Blocchi

Alice deve inviare a Bob l'importo di un bonifico destinato a Mallory: `456789`. Il messaggio viene cifrato con RC4 utilizzando una chiave segreta condivisa tra Alice e Bob. Mallory intercetta il pacchetto in transito e desidera alterare la prima cifra portandola da `4` a `9`.

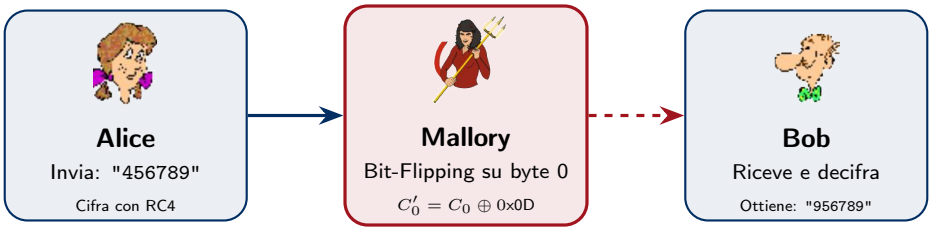


Figura 3: Attacco di Bit-Flipping su Cifrario a Flusso privo di Controllo di Integrità.

Poiché i codici ASCII delle cifre decimali condividono i primi 4 bit più significativi (`0x30`–`0x39`, ovvero binario `0011 xxxx`), modificando solo i 4 bit meno significativi vi è una probabilità pari a 10/16 che il byte decifrato rimanga una cifra decimale valida! La maschera di trasformazione per convertire `'4'` (`0x34`) in `'9'` (`0x39`) è:

$$\Delta = 0\text{x}34 \oplus 0\text{x}39 = 0\text{x}0\text{D} \tag{4}$$

Mallory modifica il primo byte del cifrato applicando lo XOR con `0x0D`:

Mallory/Bob — Attacco Bit-Flipping su Cifrario RC4

```
Testo originale di Alice: 456789
hex string is too short, padding with zero bytes to length
File numeri_tampered.enc alterato da Mallory creato con successo.
hex string is too short, padding with zero bytes to length
Testo decifrato da Bob (Bonifico manipolato): 956789
```

Spiegazione del Risultato: Bob decifra il file senza riscontrare alcun errore e legge l'importo manomesso: **956789 euro**. Il bonifico è stato fraudolentemente incrementato di 500.000 euro! Questo dimostra che **la confidenzialità non implica l'integrità**. Cifrare un messaggio senza un codice di autenticazione (MAC o modalità AEAD come AES-GCM) espone a manomissioni invisibili.

5.1.1 Cosa accade applicando il medesimo attacco su AES-128-CBC?

Mallory ripete la medesima manipolazione del primo byte su un file cifrato con AES in modalità CBC:

Mallory/Bob — Fallimento Attacco Bit-Flipping su AES-128-CBC

```
bad decrypt
806189F301000000:error:1C800064:Provider routines:ossl_cipher_unpadblock:bad decrypt:providers/
    implementations/ciphers/ciphercommon_block.c:107:
Esito decifratura CBC alterato (corruzione propagata / errore padding): 1
```

Conclusione: La modifica di un singolo byte in un blocco CBC corrompe in modo incontrollabile l'intero blocco decifrato corrispondente (effetto valanga inverso) e causa un errore di padding PKCS#7 al termine del flusso, provocando l'immediato blocco dell'applicazione con eccezione `bad decrypt`.

5.2 Esercizio 4.2: Vulnerabilità dei Pattern in Modalità ECB vs CBC

Si analizza la famigerata vulnerabilità della modalità Electronic Codebook (ECB). Alice cifra due file contenenti rispettivamente stringhe ripetute (`strastrastrastra...`) e stringhe prive di ripetizioni di periodo 8 byte.

Alice — Ispezione Blocchi Ripetuti in DES-ECB

```
Esame byte cifrati ecb_enc_1 (DES-ECB con pattern ripetuti):
          0d  79  d4  41  fc  65  03  63  0d  79  d4  41  fc  65  03  63
*
          08  6f  9a  1d  74  c9  4d  4e

Esame byte cifrati ecb_enc_2 (DES-ECB senza pattern identici a blocchi):
          b8  d8  ef  f2  1b  c2  ae  b5  a0  50  c7  08  f3  1e  ea  b2
          d5  2f  79  03  79  93  56  46  78  15  fa  89  18  1c  7a  5f
```

Analisi dei Blocchi Cifrati: Nel file cifrato `ecb_enc_1` si nota chiaramente la sequenza esadecimale ripetuta identica a blocchi di 8 byte:

`0d 79 d4 41 fc 65 03 63` ripetuta esattamente nel secondo blocco!

Poiché in ECB ogni blocco di testo in chiaro viene cifrato in modo puramente deterministico e indipendente dagli altri senza vettori di inizializzazione, **blocchi identici in ingresso producono blocchi identici in uscita**. Questa debolezza strutturale rivela la conformazione interna dei dati (come dimostrato visivamente dal celebre Tux di Linux cifrato in ECB).

Attenzione alle Trappole & Errori Frequenti

Regola Aurea di Ingegneria della Sicurezza: La modalità ECB non deve mai essere impiegata in scenari reali, ad eccezione del caso strettamente circoscritto in cui la dimensione del payload sia rigorosamente limitata a un unico singolo blocco.

5.3 Esercizio 4.3: Monitoraggio dell'Integrità Ricorsiva di Alberi di File

Alice genera una struttura di directory ramificata tramite `gen_tree.sh` e calcola l'impronta crittografica SHA-256 di tutti i file archiviando i digest nel database `hash_list.txt`.

Alice — Calcolo e Verifica Ricorsiva dell'Integrità con shasum

```
tree/sub1/config.cfg
tree/data.db
tree/sub2/system.log
tree/doc1.txt
Calcolo sha256sum di tutti i file nell'albero:
11c61ae1073b478eb8cc553a243f76ad7bd3c22293f997f07d184b02ed3eee6e  tree/sub1/config.cfg
9d12454c0fe1e1e8d1eaa4dc62d35a28954dd324d16a0c575981c3e507bb274b  tree/data.db
bac1606046d674b0a7e22bd961f3665dc5ca85d9ac01ec7181a5c56c3308058e  tree/sub2/system.log
ee86d282b39b78274174cf64272c472b0c387e3b1d6064170801dc05daefd70d  tree/doc1.txt
Verifica integrità pre-modifica:
tree/sub1/config.cfg: OK
tree/data.db: OK
tree/sub2/system.log: OK
tree/doc1.txt: OK
Manomissione non autorizzata di tree/sub1/config.cfg da parte di un intruso:
Riesecuzione verifica integrità post-manomissione (rilevazione discrepanza):
tree/sub1/config.cfg: FAILED
```

Alice — Calcolo e Verifica Ricorsiva dell'Integrità con shasum *(continua dalla pagina precedente)*

```
tree/data.db: OK
tree/sub2/system.log: OK
tree/doc1.txt: OK
shasum: WARNING: 1 computed checksum did NOT match
Codice uscita verifica integrità compromessa: 1
```

Analisi della Minaccia e Contromisure:

- **Vettore di Evasione dell'Intruso:** Se un attaccante ottiene privilegi di scrittura locali sulla macchina, dopo aver manomesso un file può semplicemente rigenerare l'intero file `hash_list.txt` per rendere invisibile l'intrusione.
- **Contromisure Efficaci:**
  1. Archiviazione dei file di integrità su server remoto protetto e inaccessibile (es. server SIEM o syslog crittografato).
  2. Impiego di **Keyed-Digest (HMAC)**: l'hash viene calcolato includendo un segreto noto solo all'amministratore, impedendo a chiunque non conosca la chiave di rigenerare le firme di integrità.
  3. Firma digitale asimmetrica del database degli hash tramite la chiave privata dell'amministratore.